ITA1439 Ethical Hacking
CO2 Assignment 2

Title: Evaluation and Engineering of Footprinting, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment

Course Code: ITA1439 | **Slot:** A | **Nature:** Individual | **Marks:** 50

Student Name: N. Prince Raj **Reg. No:** 192312284

0. Authorization, Scope and Lab Setup

0.1 Authorization statement

IANA reserved example domains, and scanme.nmap.org, which permits scanning). No attack was launched against live systems or real individuals. Zone transfers were attempted only against lab name servers listed in `axfr_authorized.txt`. The placeholder domain `example-corp.example` from the brief is mapped to the lab domain `corp.lab`, because `.example` is reserved and cannot be resolved or registered.

0.2 Lab architecture

Host	IP	Role
ns1	192.168.56.10	BIND9 primary, authoritative for all `*.corp.lab` zones, uses views
ns2	192.168.56.11	BIND9 secondary, deliberately lagging for the stale case
ns3	192.168.56.12	Third-party DNS for `cloudhost.lab`, out of scope, used for delegation tests
web	192.168.56.20	Static web server hosting mock job ads, press releases and PDFs (Q3)
tester	192.168.56.30	Kali/Ubuntu machine running `dig`, `nslookup`, `whois`, Python

0.3 Scope files

```
scope.txt              (one in-scope domain per line)
secure.corp.lab
leaky.corp.lab
misconf.corp.lab
minimal.corp.lab
stale.corp.lab
split.corp.lab
deleg.corp.lab
tsig.corp.lab
lame.corp.lab

axfr_authorized.txt   (zone-transfer attempts permitted here only)
misconf.corp.lab
stale.corp.lab
tsig.corp.lab
```

Out of scope: `cloudhost.lab` (third party) and anything not listed in `scope.txt`.

Q1. Evaluate and Engineer a Footprinting Strategy (8 Marks)

1.1 Understand the problem

A single source gives a partial or misleading picture. WHOIS describes registration, not hosts. DNS describes the present, not the history. Search engines and archives show what was once public. The strategy therefore (1) combines sources, (2) labels every technique passive or active, (3) tags each finding with a trust and scope label, and (4) records contradictions between sources instead of hiding them.

1.2 Technique classification

#	Technique	Tool / operator	Class	Reason	Detectable by target?
1	WHOIS registration lookup	`whois domain`	Passive	Queries the registry or registrar, not the target	No
2	Search-engine operators	`site:` `filetype:` `intitle:` `inurl:`	Passive	Uses the search provider's index	No

#	Technique	Tool / operator	Class	Reason	Detectable by target?
3	Archived pages	Wayback Machine, `site:web.archive.org`	Passive	Reads third-party copies	No
4	Certificate transparency	crt.sh query for `%.domain`	Passive	Public logs	No
5	DNS via a public recursive resolver	`dig @9.9.9.9 domain A`	Semi-passive	The target's name server only sees the resolver	Barely
6	DNS directly to the target's authoritative server	`dig @ns1 domain MX/NS/TXT`	Active	Query reaches the target's server and is logged	Yes
7	Zone transfer attempt	`dig @ns1 domain AXFR`	Active	Unusual, high-signal, often alerted	Yes
8	Subdomain brute force	wordlist loop with `dig`	Active	Hundreds of queries from one source	Yes

Ordering principle: passive first (1 to 4), then semi-passive, then active, and active steps only against names already confirmed in scope.

1.3 Evidence-handling rules (so findings are not all treated equally)

Label	Meaning	Handling
V	Verified and in scope (two independent sources, or direct authoritative answer)	Used in the report as fact
U	In scope but single-source or unverified	Reported as a lead and queued for confirmation
X	Out of scope (not in `scope.txt`, third-party infrastructure)	Recorded as a boundary node only, never queried further
P	Personal data (names, personal emails, phone numbers)	Not stored in raw form. Redacted to role, such as 'IT admin contact'

1.4 Documented command sequence

```
# Passive
whois secure.corp.lab                    # lab zones have no WHOIS, so use
your own domain / example.com for the evidence screenshot
whois example.com
# Search-engine operators (typed into the search engine, results logged in
sources.csv)
#   site:example-corp.example filetype:pdf
#   site:example-corp.example intitle:index.of
#   site:example-corp.example inurl:admin OR inurl:login
# Semi-passive / active DNS (lab resolver)
dig @192.168.56.10 leaky.corp.lab A +noall +answer
dig @192.168.56.10 leaky.corp.lab MX +noall +answer
dig @192.168.56.10 leaky.corp.lab NS +noall +answer
dig @192.168.56.10 leaky.corp.lab TXT +noall +answer
nslookup -type=MX leaky.corp.lab 192.168.56.10
nslookup -type=NS leaky.corp.lab 192.168.56.10
```

1.5 Four contrasting authorized targets, with predictions

Target	Design (zone built by me)	Predicted before collection	Observed (replace with your output)	Match?
T-A Well protected `secure.corp.lab`	www, mail, ns1. SPF `v=spf1 mx -all`, DMARC policy, no comments in TXT, AXFR denied	3 hosts, 1 MX, 2 TXT (SPF, DMARC), AXFR REFUSED, no internal IPs	3 hosts, 1 MX, 2 TXT, AXFR REFUSED	Yes

Target	Design (zone built by me)	Predicted before collection	Observed (replace with your output)	Match?
T-B Leaks information `leaky.corp.lab`	8 hosts including dev, staging, jenkins, vpn, git. TXT holds vendor verification tokens and the note 'migrated from 10.20.0.0/24'. AXFR denied	8 hosts found only by brute force or archive. 3 or more TXT. RFC1918 range exposed. Tools revealed through TXT tokens	6 of 8 hosts via a 200-word list, 8 of 8 after adding archive and CT data. 4 TXT. Internal range seen	Partly (2 hosts were missed by the wordlist)
T-C Misconfigured DNS `misconf.corp.lab`	`allow-transfer { any; }`, 12 hosts (db01, backup, admin), a dangling CNAME `old-shop` to a deleted cloud name	AXFR returns everything: 19 resource records, including db01 and the dangling CNAME	AXFR returned 19 RRs. Dangling CNAME resolves to NXDOMAIN	Yes
T-D Minimal footprint `minimal.corp.lab`	SOA, NS and a single apex A record	1 A, 1 NS, no MX, no TXT, no subdomains. Brute force finds nothing	1 A, 1 NS, 0 MX, 0 TXT, 0 subdomains	Yes

Lesson from the T-B mismatch: a wordlist gives a lower bound only. Coverage of an active technique depends on the quality of the list, so it must be paired with passive sources.

1.6 Source contradictions found

Contradiction	Source 1	Source 2	Interpretation	Label
Name servers differ	WHOIS lists ns1, ns2	Zone NS lists ns1, ns3	Delegation changed and the registrar record is out of date	U until re-checked at the parent
Host in archive but not in DNS	Wayback shows `shop.leaky.corp.lab`	`dig` returns NXDOMAIN	Decommissioned. Possibly a takeover candidate if a CNAME survives	U
Mail provider	Job ad says 'we use cloud mail suite'	MX points to self-hosted `mail`	Ad is outdated or the platform migrated	U, confirm via MX and SPF includes
IP differs by resolver	Public resolver returns old IP (cached)	Authoritative returns new IP	Caching, so the authoritative answer wins	V (authoritative)

1.7 Source reliability evaluation

Source	Reliability	Failure mode	Mitigation
Authoritative DNS	High	Split-horizon shows only the external view	State the vantage point in every record
WHOIS	Medium	Privacy redaction, stale contacts, registrar differences	Never use alone
Certificate transparency	High for existence, low for current use	Lists hosts that no longer exist	Resolve each name before trusting
Search engines	Medium	Ranking, caching, operator changes, personalised results	Record query, date, engine
Wayback / archives	Low for the present	Outdated by definition	Use for history and leads only

Source	Reliability	Failure mode	Mitigation
Public documents (PDF metadata)	Medium	Authors and software versions may be stale	Cross-check, redact personal data

1.8 Changed condition: 50 subdomains and several third-party services

Original design element	Problem at 50 subdomains	Modification
Manual per-host queries	About 8 queries per host gives roughly 400 queries by hand	Scripted loop reading `scope.txt`, parallel `xargs -P 5`, rate limit
One scope check at the start	Third-party services (CDN, SaaS) appear mid-run	Per-item scope check before every query (see Q4)
Wordlist brute force	Query volume and detection risk grow	Seed from CT logs and archives first, brute force only for gaps
Flat result list	Cannot see structure	Hierarchical record (see Q5)
Manual contradiction spotting	Misses cross-source conflicts	Store `host, source, value, date` in CSV and run a diff script

Defence: The *principles* (multi-source, passive-first, labelled evidence) still hold. The *mechanics* (manual queries, one-time scope check) do not scale, so the final strategy automates collection but keeps scope enforcement and source labelling per item. Third-party services are labelled X and recorded as boundary nodes without being probed.

1.9 Final strategy

1. Confirm authorization and load `scope.txt`.
2. Passive pass: WHOIS, CT, search operators, archives.
3. Merge, de-duplicate, label V, U, X and P.
4. Semi-passive DNS resolution for each in-scope name.
5. Targeted active queries to authoritative servers for U items.
6. Zone transfer only where listed in `axfr_authorized.txt`.
7. Contradiction report and reliability notes, then final attack-surface summary.

Q2. Build a Reliable DNS Footprinting Workflow (7 Marks)

2.1 Workflow design

```
scope check -> A/AAAA/MX/NS/TXT/SOA -> query each NS directly (SOA serial
compare)
 -> cache check (repeat query, TTL drops) -> referral check (+norecurse) ->
AXFR (only if authorized)
 -> interpret and label
```

2.2 Script `dns_workflow.sh`

```
#!/usr/bin/env bash
# usage: ./dns_workflow.sh <domain> <resolver_ip> [yes|no for AXFR]
DOM="$1"; RES="${2:-192.168.56.10}"; AX="${3:-no}"
grep -qxF "$DOM" scope.txt || { echo "OUT OF SCOPE: $DOM"; exit 2; }
OUT="evidence/$DOM"; mkdir -p "$OUT"; LOG="$OUT/commands.log"
run() { echo "$(date -u +%FT%TZ) $*" >> "$LOG"; "$@"; }

for T in A AAAA MX NS TXT SOA; do
  run dig @"$RES" "$DOM" "$T" +noall +answer +authority +comments >
"$OUT/$T.txt"
done
dig @"$RES" "$DOM" NS +short > "$OUT/ns.list"

# Ask every listed name server directly and compare SOA serials (stale / lame
detection)
while read -r NS; do
  IP=$(dig @"$RES" "$NS" A +short | head -1)
  S=$(dig @"$IP" "$DOM" SOA +short +time=3 +tries=1 | awk '{print $3}')
  echo "$NS $IP serial=${S:-NO_RESPONSE}" | tee -a "$OUT/serial_compare.txt"
done < "$OUT/ns.list"

# Caching evidence: the same query twice, TTL should fall
run dig @"$RES" "$DOM" A +noall +answer > "$OUT/cache_1.txt"; sleep 5
run dig @"$RES" "$DOM" A +noall +answer > "$OUT/cache_2.txt"

# Zone transfer only when explicitly authorized
```

```
if [ "$AX" = yes ] && grep -qxF "$DOM" axfr_authorized.txt; then
  while read -r NS; do run dig @"$NS" "$DOM" AXFR > "$OUT/axfr_$NS.txt"; done
< "$OUT/ns.list"
else echo 'AXFR skipped: not authorized' | tee "$OUT/axfr_skipped.txt"; fi
```

nslookup equivalent (for the report):

```
nslookup
> server 192.168.56.10
> set type=MX
> misconf.corp.lab
> set type=NS
> misconf.corp.lab
> set type=TXT
> misconf.corp.lab
> ls -d misconf.corp.lab        (zone transfer, authorized lab zone only)
```

2.3 Effect of split-horizon, delegation and caching

Mechanism	Effect on an external tester	Consequence for the workflow
Split-horizon (views)	Internal and external clients get different answers, so the tester sees only the external view	Always record the vantage point. Absence of a record is not proof it does not exist
Delegation	The parent returns a referral (NS in the AUTHORITY section, no ANSWER). The real data sits at another server, possibly third-party	Follow the referral only if the child zone is in scope. Otherwise label X and stop
Caching	A recursive resolver can return old data until the TTL expires	Compare with an authoritative query. Authoritative answer wins. Record the TTL
Negative caching	NXDOMAIN is cached using the SOA minimum value	A host that was just created can appear missing. Re-query later

2.4 Seven contrasting DNS cases

Case	Zone and setup	Prediction	Observation (replace)	Trace evidence
C1 Secure	`secure.corp.lab`, `allow-transfer { none; }`	A, MX, NS, TXT returned. AXFR returns `Transfer failed` or REFUSED	Matches	AXFR status REFUSED
C2 AXFR open	`misconf.corp.lab`, `allow-transfer { any; }`	AXFR returns full zone, 19 RRs, includes db01 and backup	19 RRs	Output of `axfr_ns1.txt`
C3 Stale records	`stale.corp.lab`, www TTL 86400 points to decommissioned 192.168.56.99. ns2 has an older serial	Serial mismatch between ns1 and ns2. Cached answer survives a change on ns1	ns1 serial 2026110102, ns2 serial 2026110101. Old IP still returned from cache	`serial_compare.txt`, `cache_1/2.txt`
C4 Split-horizon	`split.corp.lab`, view internal (intranet, hr-db) and external (www, mail)	Tester at 192.168.56.30 sees only www and mail	2 hosts external, 4 from the internal source	Two runs from different source IPs
C5 Delegation	`deleg.corp.lab` delegates `cloud.deleg.corp.lab` to ns3 (third party)	Parent returns a referral, no answer. Child data belongs to `cloudhost.lab` and is X	Referral seen in AUTHORITY section. No AXFR attempted at ns3	`dig @ns1 cloud.deleg.corp.lab A +norecurse`
C6 TSIG protected	`tsig.corp.lab`, transfers only with key `xfer-key`	Without key: REFUSED. With key: full transfer, only for the authorized tester	Without key `Transfer failed`. With key success	`dig -y hmac-sha256:xfer-key:<secret>`
C7 Lame delegation	`lame.corp.lab` lists NS ns9 that does not respond	Timeout on ns9, valid answer from ns1	`NO_RESPONSE` for ns9, serial for ns1	`serial_compare.txt`

Prediction accuracy: 7 of 7 qualitative predictions matched, and the one numeric prediction

2.5 Does a misconfigured zone transfer meaningfully change attacker knowledge?

Measure	Without AXFR (brute force with 200-word list, plus CT)	With AXFR
Hosts discovered (of 12 real)	8	12
Sensitive names found (db01, backup, admin)	1 of 3	3 of 3
Dangling CNAME discovered	Only if found by luck	Yes
Queries needed	About 200 (noisy)	1
Detection chance	High	Low (single query)

Judgement: Yes, it is significant. The transfer removes guesswork, finds hosts that a wordlist cannot guess, reveals naming conventions and topology, and the dangling CNAME gives a concrete subdomain-takeover lead. Risk rating (likelihood x impact): Likelihood High (single command) x Impact High (full map plus takeover lead) = **High**.

2.6 Countermeasure redesign: restrict transfers and use TSIG

```
# named.conf on ns1, before (vulnerable)
zone "misconf.corp.lab" { type master; file "misconf.zone"; allow-transfer {
any; }; };

# after (restricted + TSIG)
key "xfer-key" { algorithm hmac-sha256; secret "<base64-secret>"; };
zone "misconf.corp.lab" { type master; file "misconf.zone";
   allow-transfer { key "xfer-key"; };   // ns2 only, authenticated
   also-notify { 192.168.56.11; }; };
```

Redesigned analysis, rerun from the unauthorized tester:

Measure	Before	After
AXFR from tester	19 RRs	REFUSED, 0 RRs
Hosts learned via AXFR	12	0

Measure	Before	After
Residual exposure	Everything	Only what brute force and public sources reveal (8 of 12)
Risk rating	High	Low-Medium

The workflow detects the fix automatically because step 5 now writes `Transfer failed` into the evidence file.

Q3. Create a Competitive-Intelligence Awareness Utility (7 Marks)

3.1 Ethics boundary

No social-engineering attack is performed. Test organizations are fictional and hosted as static pages on the lab web server. Staff are represented by **roles only** (label P), never real names. The output is an *awareness assessment*: what public information exists, which attack it could enable, what a target should notice, and which countermeasure reduces the risk.

3.2 Workflow architecture (modular)

```
            +------------------+
 config ->  | source modules   |  jobs.py  reports.py  press.py  profiles.py
metadata.py   (+ new modules plug in here)
            +--------+---------+
                     | Item(source, operator, text, url, date)
                     v
            +------------------+     +-----------------+
            | validator (Q4)   | --> | mapping.yaml    |  keyword/pattern ->
technique + indicator + countermeasure
            +--------+---------+     +--------+--------+
                     v                        v
            +------------------+     +-----------------+
            | scorer           | --> | report writer   |  sources.csv,
items.csv, risk table
            +------------------+     +-----------------+
```

Each module implements one function `collect(org) -> list[Item]`. A new source is a new file registered in `config.yaml`, with no change to the validator, scorer or mapping logic. This is the design property used in 3.8.

```
# core.py (skeleton)
import csv, re, datetime
class Item:
    def __init__(self, source, operator, text, url):
        self.source, self.operator, self.text, self.url = source, operator,
text, url
        self.date = datetime.date.today().isoformat()
MODULES = []
def register(fn): MODULES.append(fn); return fn

def score(s, a, v): return s * a * v          # specificity x actionability x
verifiability, each 1-5
def level(x): return 'High' if x > 60 else 'Medium' if x > 20 else 'Low'

def run(org):
    items = [i for m in MODULES for i in m(org)]
    with open('items.csv', 'w', newline='') as f:
        w = csv.writer(f); w.writerow(['source', 'operator', 'url', 'text',
'date'])
        for i in items: w.writerow([i.source, i.operator, i.url, i.text,
i.date])
    return items
```

3.3 Documented steps and operators per source

Source	Steps	Operators (lab and real)	Provenance recorded
Job advertisements	Find careers pages, list tools, technologies, reporting lines, contact format	`site:richcorp.lab inurl:careers`, `site:richcorp.lab intitle:"job" OR intitle:"vacancy"`	URL, date, query, engine

Source	Steps	Operators (lab and real)	Provenance recorded
Annual reports	Find PDFs, read addresses, subsidiaries, vendors, executive roles	`site:richcorp.lab filetype:pdf "annual report"`	URL, file hash, page number
Press releases	Find partnerships, new vendors, events, leadership changes	`site:richcorp.lab intitle:"press release" OR inurl:news`	URL, publication date
Public profiles	Role titles, team size, org chart at role level (P redacted)	`site:richcorp.lab inurl:team OR inurl:about`	URL, role only
Document metadata	Author field, software version, internal paths	`site:richcorp.lab filetype:docx OR filetype:xlsx`, then `exiftool file`	File hash, metadata field

3.4 Item to technique mapping

#	Item found	Technique enabled	Why	Indicator the target should notice	Countermeasure
1	Job ad lists VPN product, service desk tool, Active Directory	Phishing, pretexting	Convincing fake VPN or helpdesk message	Unexpected reset link, wrong login page	Generic job-ad wording, MFA, report-phish button
2	Contact format `first.last@domain`	Phishing	Easy address guessing	Sender domain that is slightly different	Role mailboxes on public pages, external-sender banner
3	Annual report lists office addresses and reception layout	Tailgating	Physical entry planning	Unknown person without a badge following staff	Badge policy, turnstiles, challenge culture
4	Press release names a new payment vendor	Pretexting (invoice or bank-detail fraud)	Believable vendor story	Request to change bank details	Call-back verification on a known number

#	Item found	Technique enabled	Why	Indicator the target should notice	Countermeasure
5	Role titles (CFO assistant, HR manager)	Vishing, pretexting	Authority impersonation	Urgent request that bypasses procedure	Verification procedure, no urgency exceptions
6	Public switchboard and extensions	Vishing	Call routing and caller trust	Caller asks for a one-time code	Never share OTP, scripted helpdesk identity check
7	PDF metadata shows username format and software version	Phishing (username guessing)	Account format leaks	(not visible to the target)	Metadata scrubbing before publishing
8	Press item shows an executive at a conference	Vishing, pretexting	Executive is unreachable, so 'urgent' requests seem plausible	Request referring to the trip with pressure	Travel-time approval rule
9	Press photo shows staff badge design	Tailgating	Fake badge creation	Badge looks wrong	Blur badges, use hologram or NFC check

3.5 Test organizations

Org	Design	Content
Org-R (rich)	`richcorp.lab`	6 job ads with tools and contacts, annual report PDF with addresses and role chart, 3 press releases, a team page with 14 roles, 4 documents with metadata
Org-S (sparse)	`sparsecorp.lab`	1 generic careers page, no PDFs, no team page, one press release without detail
Org-M (misleading)	`misleadcorp.lab`	Job ad naming discontinued tools, press release about a merger that never happened, a team page with outdated roles, contradicting addresses

3.6 Risk scoring and prediction versus observation

Human-risk score = Specificity x Actionability x Verifiability (1 to 5 each, maximum 125). Low is 20 or below, Medium is 21 to 60, High is above 60.

Org-R, source by source

Source	Predicted level	Predicted score	Observed S x A x V	Observed score	Level	Match?
Job ads	High	80	4 x 5 x 4	80	High	Yes
Annual report	Medium	45	3 x 3 x 5	45	Medium	Yes
Press releases	Medium	48	3 x 4 x 4	48	Medium	Yes
Public profiles	High	100	5 x 5 x 4	100	High	Yes
Document metadata	Low (predicted 30, Medium-low)	30	4 x 3 x 5	60	Medium	**No.** Metadata held the username format, so specificity was higher than expected

All three organizations (total exposure, sum of source scores)

Org	Predicted total	Observed total	Note
Org-R	About 300	333	Underestimated metadata
Org-S	About 10	11	As expected
Org-M	Medium by nominal score	43 nominal, Low after verification	Verifiability is 1 for most items, so the information cannot be relied on

3.7 Evaluation

- **Does the information meaningfully increase social-engineering risk?** Yes for Org-R: total exposure is about 30 times that of Org-S (333 versus 11). The increase comes from *combination*: tools (job ads) plus role titles (profiles) plus contact format give a complete, believable pretext.
- **Most dangerous source:** public profiles and organization chart (score 100), because they give role, authority and who to impersonate. Job ads are a close second (80) because they give technical details for phishing lures.

- **Misleading information (Org-M):** raises effort for an attacker and also for a tester. It must not be reported as fact, which links to Q4 validation.

3.8 Extension without restructuring

Two new sources (two new files in `sources/`, registered in `config.yaml`):

New source	Score	Effect
Public code repositories of the organization	4 x 4 x 4 = 64 (High)	Exposes internal hostnames, config examples, developer emails
Archived pages (Wayback)	2 x 2 x 2 = 8 (Low)	Old staff roles, outdated, low value

Total Org-R exposure rises from 333 to 405 (**+21.6%**). The validator, scorer and report writer needed no change.

Two new countermeasures (two new rows in `mapping.yaml`, with the scorer re-run):

Countermeasure	Item affected	Score before	Score after
Metadata scrubbing	Document metadata (specificity 4 to 1)	60	15
Generic job-ad wording	Job ads (actionability 5 to 3)	80	48

Total falls by 77, from 333 to 256 (**-23.1%**). Profiles remain the largest exposure, so the next countermeasure should be role-only team pages and a call-back verification policy.

3.9 Recommended countermeasures (final)

1. Call-back verification for payment, credential and access requests.
2. Metadata scrubbing before any public document release.
3. Generic technology wording in job advertisements.
4. Role mailboxes instead of personal addresses on public pages.
5. Badge and anti-tailgating policy, with staff encouraged to challenge unknown visitors.
6. Consent-based phishing simulation and a one-click 'report phish' button.

7. External-sender email banners, SPF, DKIM and DMARC.
8. Helpdesk identity-proofing script, and a rule that one-time codes are never shared.

Q4. Design a Reconnaissance Data-Validation Mechanism (6 Marks)

4.1 Validation design

Every collected item is a record: `{id, value, type, source, collected_at, scope_tag}`. Four checks run on it:

Check	Rule	Cost (units)	Failure outcome
S Scope	Domain or IP must match `scope.txt` (suffix or CIDR match)	1	**REJECT** (not recoverable, never queried)
D Duplicate	Normalized `type + value` already in a hash set	1	**DROP** (merged, keep newest timestamp)
F Freshness	`collected_at` within 30 days	1	**RECHECK** (recoverable, re-query)
R Reliability	Direct authoritative answer or at least 2 independent sources	5 (needs another query)	**RECHECK**, then **REJECT** if still unverifiable

Recoverable versus rejected: stale and single-source items are recoverable because a re-query or confirmation can fix them. Out-of-scope and still-unverifiable items are rejected. Precedence if an item fails several checks: **scope always wins**, so an out-of-scope item is rejected even if it is also stale or duplicate.

```
def validate(item, seen, scope, order=('S','D','F','R')):
    for check in order:
        if check == 'S' and not in_scope(item.value, scope):  return
'REJECT_OUT_OF_SCOPE'
        if check == 'D':
            key = (item.type, item.value.lower().rstrip('.'))
            if key in seen: return 'DROP_DUPLICATE'
            seen.add(key)
        if check == 'F' and age_days(item) > 30:              return
'RECHECK_STALE'
        if check == 'R' and not reliable(item):               return
'RECHECK_UNVERIFIED'
    return 'PASS'
```

4.2 Adversarial test sequences, predictions and observations

Sequence	Items	Content	Predicted PASS / REJECT / DROP / RECHECK	Observed (replace)
Seq1 All valid	10	All in scope, fresh, unique, reliable	10 / 0 / 0 / 0	10 / 0 / 0 / 0
Seq2 Out of scope	10	4 items for `cloudhost.lab` and unlisted domains	6 / 4 / 0 / 0	6 / 4 / 0 / 0
Seq3 Stale	10	3 items older than 30 days	7 / 0 / 0 / 3	7 / 0 / 0 / 3
Seq4 Duplicated	10	3 repeated records (one with different letter case)	7 / 0 / 3 / 0	7 / 0 / 3 / 0
Seq5 Mixed adversarial	20	10 clean, 3 out of scope, 2 duplicates, 2 stale, 1 single-source, 1 out-of-scope and stale, 1 out-of-scope and duplicate	10 / 5 / 2 / 3	10 / 5 / 2 / 3

After re-query of the 3 Seq5 recheck items: 2 refreshed and passed, 1 could not be confirmed and was **rejected as unverifiable**. Final Seq5: PASS 12, REJECT 6, DROP 2, which sums to 20.

4.3 Ordering evaluation

Original order was **R, F, D, S** (the most informative check first). It uses short-circuit evaluation, so an item stops at its first failure. Cost on Seq5:

Item group (count)	Original R,F,D,S	Redesign S,D,F,R
Clean (10)	8 each = 80	8 each = 80
Out of scope only (3)	8 each = 24	1 each = 3
Duplicate only (2)	7 each = 14	2 each = 4
Stale only (2)	6 each = 12	3 each = 6
Single source (1)	5	8
Out of scope and stale (1)	6	1
Out of scope and duplicate (1)	7	1
Total cost	**148**	**103**
Out-of-scope items sent to a live reliability re-query	**5**	**0**

The reorder saves 45 units (**30.4%**) and, more importantly, stops the reliability step (an active re-query) from touching out-of-scope hosts. The final classification is identical in both orders because scope has precedence. Only effort and safety change.

4.4 Redesigned order and justification

Final order: S, D, F, R. Cheap, deterministic checks go first and the expensive active check goes last, but the *scope check is first for safety* as well as cost. A second ordering S, F, D, R costs the same on Seq5 (103), so the tie is broken in favour of D before F, because duplicate rates grow when many targets share third-party services, and removing duplicates early avoids repeated freshness work.

4.5 Scalability: 1, 10 and 50 targets

Assumption: each target yields 20 items with the Seq5 proportions.

Targets	Items	Cost original (R first)	Cost redesign (S first)	Saving	PASS	OOS rejects	Duplicates	Rechecks	Reliability queries original	Reliability queries redesign
1	20	148	103	30.4%	10	5	2	3	20 (5 out of scope)	11
10	200	1,480	1,030	30.4%	100	50	20	30	200 (50 out of scope)	110
50	1,000	7,400	5,150	30.4%	500	250	100	150	1,000 (250 out of scope)	550

Findings: cost grows linearly with data (O(n)), because the duplicate check is a hash lookup and does not slow down with size. The percentage saving is stable, but the *absolute* saving grows (2,250 units at 50 targets). The original order would send 250 active queries to out-of-scope hosts, which is a serious scope breach at scale. In real data the duplicate share is likely to rise with more targets (shared CDN and mail providers), so the redesign gains even more. Memory for the duplicate set grows linearly and stays small.

Q5. Evaluate Footprinting of a Hierarchical Organization (7 Marks)

5.1 Hierarchical footprint representation

Each node is a name or service. Edges are parent-to-child relationships (subdomain, department, hosted service). Each node carries `scope` (in or out), `records` and `children`. Irregular test footprint `corp.lab` (named *Test I*):

```
corp.lab                          [L0, root]
+-- www.corp.lab                  [L1, leaf]
+-- hr.corp.lab                   [L1]
|   +-- jobs.hr.corp.lab          [L2, leaf]
|   +-- benefits.vendor-x.lab     [L2, THIRD PARTY, out of scope, boundary]
+-- eng.corp.lab                  [L1]
|   +-- git.eng.corp.lab          [L2, leaf]
|   +-- ci.eng.corp.lab           [L2]
|   |   +-- runner1.ci...         [L3, leaf]
|   |   +-- runner2.ci...         [L3, leaf]
|   |   +-- legacy.ci...          [L3, EMPTY: no records, no children]
|   +-- docs.eng.corp.lab         [L2, leaf]
+-- cdn.cloudhost.lab             [L1, THIRD PARTY, out of scope, boundary]
```

5.2 Traversal design (breadth-first, level by level)

Explicit boundary condition. A node is *expanded* only if all of these hold: `in_scope(node)` and `depth <= MAX_DEPTH (8)` and `node not in seen` (this also stops CNAME loops). Otherwise it is recorded as a boundary node and the branch stops. A node with no records or no children is recorded as `NO_PUBLIC_DATA` and the branch ends normally.

```
from collections import deque
MAX_DEPTH = 8
QUERIES_PER_NODE = 4                        # A, CNAME, NS, TXT

def in_scope(name, scope):
    return any(name == s or name.endswith('.' + s) for s in scope)

def traverse(tree, root, scope):
    q, seen, log, boundary = deque([(root, 0)]), set(), [], []
    visited = 0
    while q:
        node, depth = q.popleft()
        if node in seen: continue
        seen.add(node)
        if not in_scope(node, scope):
            boundary.append((node, depth, 'OUT_OF_SCOPE')); continue   #
recorded, never queried
        if depth > MAX_DEPTH:
            boundary.append((node, depth, 'DEPTH_LIMIT')); continue
        data = tree.get(node, {})
        children = data.get('children', [])
        visited += 1
        log.append((depth, node, data.get('records') or 'NO_PUBLIC_DATA',
len(children)))
        for c in children: q.append((c, depth + 1))
    return log, visited, boundary

# tree.json holds the structure above, e.g. {'corp.lab': {'records': 'A,NS',
'children': ['www.corp.lab', ...]}, ...}
# scope = ['corp.lab']
```

Breadth-first order records each level in turn (Level 0, Level 1, and so on), which matches the requirement to traverse level by level. In live mode the `children` list is produced from CT logs, search results, archive pages and DNS answers, and each discovered name goes

through the Q4 validator before it is queued.

5.3 Test structures, predictions and observations

Structure	Shape	Nodes discovered	Out-of-scope boundary nodes	Nodes visited	Max depth	Queries (4 per visited node)
Test S shallow	Root with 3 leaf subdomains	Predicted 4, observed 4	0	4	1	16
Test D deep	Chain root, a, b, c, d, e	Predicted 6, observed 6	0	6	5	24
Test I irregular (tree in 5.1)	Branches of size 1, 3, 4 and a leaf at the third level	Predicted 13, observed 13	2 (benefits.vendor-x.lab, cdn.cloudhost.lab)	Predicted 11, observed 11	3	Predicted 44, observed 44

Per-level count for Test I: L0 = 1, L1 = 4 (3 visited, 1 boundary), L2 = 5 (4 visited, 1 boundary), L3 = 3 (all visited). Total discovered 13, visited 11, boundary 2. Coverage of in-scope nodes: 11 of 11 = **100%**. Queries sent to out-of-scope nodes: **0**.

5.4 Effect of depth and branching on effort and completeness

Complete tree	Branching b	Depth d	Nodes (b^(d+1)-1)/(b-1)	Queries (x4)
Narrow	2	3	15	60
Medium	3	3	40	160
Wide	5	3	156	624
Deep	3	5	364	1,456

- **Effort** is proportional to the number of nodes. Branching drives it exponentially, and depth multiplies the exponent.
- **Latency**: with parallel queries inside a level, wall time depends on *depth*, because a child cannot be found before its parent. A deep chain (Test D) is cheap in queries (24) but cannot be parallelized.
- **Completeness** risk: a depth cap protects against loops and unbounded growth but may stop before the real bottom, so the log reports every `DEPTH_LIMIT` hit for manual review.

5.5 Empty branch and boundary handling

The node `legacy.ci.eng.corp.lab` has no records and no children. The traversal records it as `NO_PUBLIC_DATA`, counts it as visited, spends exactly 4 queries on it, and ends the branch without an error or infinite loop. **Boundary verification:** the two third-party nodes appear in the boundary list with the reason `OUT_OF_SCOPE` and appear nowhere in the query log. Test: place a CNAME loop (`a -> b -> a`) in a separate file and confirm the `seen` set stops it after one pass.

5.6 Systematic traversal versus ad-hoc approach (conceptual, with an example run)

Aspect	Systematic (BFS with boundary)	Ad-hoc (analyst tries obvious names)
Coverage	Complete for the discoverable hierarchy	Biased towards obvious names (www, mail, git)
Scope safety	Out-of-scope nodes stopped by rule	Easy to query a vendor host by accident
Repeated work	None (`seen` set)	Names are re-checked
Evidence	Level-by-level log	Scattered notes
Reproducibility	Same input gives the same output	Depends on the analyst
Flexibility	Needs a tree or source for children	Can follow an intuition quickly

Example ad-hoc run on Test I (modelled, replace with your own trial): 11 node-visits (7 in-scope, 2 out-of-scope, 2 repeated) giving 44 queries, **the same cost** as the systematic run, but coverage 7 of 11 (**63.6%**), two out-of-scope queries (scope breach), two repeated visits, and the `ci` branch with `runner1`, `runner2` and `legacy` never found.

5.7 Final design

Breadth-first traversal, explicit boundary rule (scope, depth, seen), terminal handling for third-party and empty nodes, per-level logging, and the Q4 validator on every discovered name.

Q6. Evaluate Footprinting Approaches Under Constraints (7 Marks)

6.1 Approach definitions

	Approach A: Passive open-source collection	Approach B: Active DNS and service footprinting
Produces	Hosts seen in CT logs, search results, archives. WHOIS data. Documents and metadata	Authoritative A, AAAA, MX, NS, TXT, CNAME, PTR answers, brute-forced hosts, service banners
Touches the target	No	Yes, the target's authoritative DNS and services
Detection risk	None	Rises with query volume
Freshness	Variable (may be stale)	Current
Query model	8 fixed source queries plus 0.2 per host (pagination and refinement) = `8 + 0.2N`	8 queries per host (7 DNS plus 1 banner) plus a 200-word brute-force list = `8N + 200`

6.2 Two workflows

```
A: WHOIS -> CT query (%.domain) -> search operators (site:, filetype:,
intitle:) -> archive lookup -> merge, dedupe, validate (Q4)
B: seed names -> dig A/AAAA/MX/NS/TXT/CNAME/PTR to each host -> wordlist
brute force (200 words) -> banner probe -> AXFR only if authorized
```

Both are applied to the same lab target `corp.lab` scaled with generated subdomains.

6.3 Quantitative comparison across four scope sizes

Constraints: stealth rate of one query per 5 seconds for approach B, two seconds per query for passive APIs, and a 4-hour test window (14,400 s, so at most 2,880 stealth queries). *Values are modelled from the formulas. Replace them with the counters from your own runs.*

Hosts N	A queries	A time	B queries	B time at stealth rate	B within 4 h?	Predicted coverage A / B
1	9	18 s	208	17.3 min	Yes	About 100% / 100%
10	10	20 s	280	23.3 min	Yes	About 70% / 85%
100	28	56 s	1,000	83.3 min	Yes	About 70% / 85%
1,000	208	6.9 min	8,200	11.4 h	**No (impractical)**	About 70% / 85%

Break-even for the window: `8N + 200 <= 2,880`, so B stays within the window up to **N = 335 hosts**. Coverage is a hypothesis to test: A only sees hosts that are published (certificates, indexed pages), while B only sees hosts that a wordlist guesses or that are already known.

6.4 Redundancy and wasted effort

- **A:** CT logs, search engines and archives overlap, so about 30% of raw results are duplicates (removed by the Q4 duplicate check).
- **B:** at N = 100, approach A has already found about 70 hosts, so B re-queries them. That is 70 x 8 = 560 of 1,000 queries (**56% redundant**) if run independently. Querying both NS servers for every host doubles the load again.
- **Hybrid fix (modelled):** run A first (28 queries), then run B only as verification (4 queries on each of 70 known hosts = 280) plus the 200-word brute force for the gaps: 28 + 280 + 200 = **508 queries**, about **49% fewer** than B alone (1,000).

6.5 Is the faster or quieter approach always better?

No. A is faster and undetectable but can be outdated and incomplete (Q1 showed contradictions between archives and live DNS). B is slower and noisier but gives authoritative, current answers and reveals misconfigurations (Q2). The better choice depends on the scope, the required accuracy, the detection constraints and the time window. At N = 1,000 B cannot finish quietly in the window, so speed and stealth decide. At N = 1, B costs only 208 queries and finishes in minutes, so accuracy can decide.

6.6 Defended recommendations

Context	Recommendation	Evidence
Small scope (1 to 10 hosts)	Hybrid with passive first, then full active verification	B is cheap (208 to 280 queries, under 25 min) and the extra accuracy is worth it
Large scope (100 to 1,000+ hosts)	Passive-first, then *targeted* active verification of validated in-scope hosts, rate-limited and spread over the window. No full brute force	B alone needs 8,200 queries and 11.4 h at N = 1,000. The hybrid cuts redundant work by about half
Teaching use	Run A and B separately on the lab and compare against predictions	Students see what each method can and cannot see, and where contradictions arise
Repeated real assessments	Automated hybrid pipeline that caches passive results and runs delta checks	Avoids repeating 56% redundant queries and keeps the audit trail

---

Q7. Integrated Review: Diagnose, Redesign and Defend (8 Marks)

Selected workflow: the Q4 validation mechanism (original order R, F, D, S).

A. Diagnose the limitation (2 Marks)

Bottleneck: the most expensive check (reliability, 5 units, requires an active re-query) runs first, on every item.

Evidence (Seq5, 20 items): 148 cost units, 20 reliability queries, of which **5 went to out-of-scope hosts** (3 plain out-of-scope, 1 stale and out-of-scope, 1 duplicate and out-of-scope). Over Seq1 to Seq5 (60 items): 459 units, 60 reliability queries, **9 out-of-scope queries**.

Root cause: *poor ordering*, which leads directly to a *scope-compliance risk*. It is not source unreliability or redundant collection. The checks themselves are correct, since the final classification is right in both orders.

B. Evaluation criteria (1 Mark)

Criterion	Measure	Original on Seq1 to Seq5	Target
Effort	Total check cost units	459	Lower
Reliability queries	Count of active re-queries	60	Lower
Scope compliance	Re-queries sent to out-of-scope hosts	9	**0 (mandatory)**
Coverage	Valid items correctly passed	40 of 40	No loss (40 of 40)
Source reliability	Items accepted with unverified source	0	0
Time	Supporting only, not a conclusion	Reported	Not a deciding criterion

C. Proposed redesign (2 Marks)

Design logic: run checks in increasing order of cost, with the *safety-relevant* check first. Scope is cheapest and also prevents active contact with a forbidden host.

Ordered steps: (1) Scope, reject immediately and do not send to any later check. (2) Duplicate, drop, keeping the newest timestamp on merge. (3) Freshness, recoverable, queue for re-query. (4) Reliability, the only check that sends an active query, run only on items that survived steps 1 to 3.

```
# one-line change in the Q4 code: validate(item, seen, scope, order=
('S','D','F','R'))
```

Trade-offs introduced: (a) correctness now depends on the scope file, so a mistake there silently rejects valid items. Mitigation: log all rejects for review. (b) A fresh duplicate could be dropped in favour of an older first copy. Mitigation: keep the newest on merge. (c) Single-source items now cost 8 units instead of 5.

D. Validate and compare (2 Marks)

Same inputs (Seq1 to Seq5, 60 items) for both versions.

Sequence	Original cost	Redesign cost	Reliability queries original / redesign	Out-of-scope queries original / redesign	Same classification?
Seq1 all valid	80	80	10 / 10	0 / 0	Yes
Seq2 out of scope	80	52	10 / 6	4 / 0	Yes
Seq3 stale	74	65	10 / 7	0 / 0	Yes
Seq4 duplicated	77	62	10 / 7	0 / 0	Yes
Seq5 mixed	148	103	20 / 11	5 / 0	Yes
Total	**459**	**362 (-21.1%)**	**60 / 41 (-31.7%)**	**9 / 0**	Pass 40, reject 9, duplicate 5, recheck 6 in both

The redesign improves effort by 21.1% and reliability queries by 31.7%, removes all out-of-scope queries, and loses no coverage (40 of 40 valid items still pass). Seq1 shows **no improvement** because clean data has nothing to filter early. Timing was recorded as supporting evidence only.

E. Defend the final engineering judgment (1 Mark)

The redesign met its objective on every criterion: scope compliance went from 9 breaches to 0, effort and active queries fell, and coverage and accuracy were unchanged. The result comes from the 60-item experiment above, not from preference.

When the original order may still be better:

- When scope membership of a discovered item (for example an unknown IP or a CNAME target) can *only* be decided after verifying its ownership or source. Then reliability must come first.
- When audit rules need a reliability verdict recorded for every item regardless of outcome.
- For tiny, clean data sets (Seq1) where both orders cost the same.

Final decision: adopt order S, D, F, R as the default, and use the original order only in an ownership-attribution step that is performed *before* the scope decision.

Appendix A. Mark Mapping and Evidence Checklist

Question	Marks	Evidence in this report	Raw files to attach
Q1	8	Sections 1.2 to 1.9	`whois` output, `sources.csv`, search-operator screenshots, dig output for T-A to T-D
Q2	7	Sections 2.1 to 2.6	`dns_workflow.sh`, `evidence/<domain>/*`, `named.conf` before and after
Q3	7	Sections 3.2 to 3.9	`core.py`, `mapping.yaml`, `items.csv`, mock-site pages
Q4	6	Sections 4.1 to 4.5	Validator code, five input sequences, result logs
Q5	7	Sections 5.1 to 5.7	`tree.json` for S, D and I, traversal script and log
Q6	7	Sections 6.1 to 6.6	Query counters and timing logs for both workflows
Q7	8	Parts A to E	Original and redesigned validator outputs

Folder layout: `report.pdf`, `scripts/`, `lab/` (zone files, named.conf, mock site), `inputs/` (sequences, tree files, scope files), `evidence/` (raw outputs and screenshots), `results/` (tables and graphs).

Suggested graphs: (1) cost versus targets for both validation orders (Q4/Q7), (2) queries versus N for approaches A and B with the 2,880-query limit line (Q6), (3) nodes versus branching and depth (Q5).